MAL • DATA PLATFORM • DELIVERABLE 3 OF 3

Customer 360 & CAR: Identity Resolution SQL & Pseudocode

Runnable-shape SQL for the deterministic waterfall and the SCD2 survivorship merge, plus Glue/PySpark pseudocode for the one step – probabilistic matching – that genuinely needs a graph engine instead of a single query.

DELIVERABLE 3 OF 3 — SQL & PSEUDOCODE

Companion Documents	Dialect	Status	Scope
#1 Architecture #2 ERD	Redshift SQL & PySpark (Glue)	Draft for review	Identity resolution only

How to Read This Code

This deliverable is scoped narrowly to identity resolution – the matching, clustering, and survivorship logic behind Section 2 of **Deliverable 1**. It does not include the PII masking policy or the CAR population query; those belong to different requirements and would only dilute this one's focus. Every table and column name here matches **Deliverable 2 (the ERD)** exactly, so the two can be read side by side.

The worked example threaded through this code – a customer named Amina, whose CRM, core banking, and Amplitude records disagree on email and name formatting but share a phone number – is the same example introduced in Deliverable 1, Section 1. Seeing the same records resolve step by step here, in actual SQL, is meant to make the logic concrete rather than abstract.

SQL is written against Redshift's current surface, including native `MERGE` and `FILTER`-clause aggregates. The one step that is a poor fit for pure SQL – scoring similarity between every unmatched pair – is shown as Glue/PySpark pseudocode instead, and Section 3 explains why that split is deliberate rather than an inconsistency.

1 Staging: Source-Aligned Tables, PII Already Tokenized

Every field that could identify a person is tokenized at landing time – a deterministic, keyed HMAC-SHA256, key held in KMS – before any matching logic ever runs. Nothing downstream of this point, including the matching engine itself, ever touches a raw national ID, phone number, or email.

```
-- ================================================================
-- Bronze -> Silver staging: one table per source. Matching, masking, and
-- the CAR all operate on tokens from this point forward -- never on a
```

```
-- raw identifier.
-- ========================================================================

CREATE TABLE stg_core_customer (
   core_customer_id VARCHAR(40) NOT NULL, -- core banking's native CIF
   national_id_token VARCHAR(64), -- HMAC-SHA256(national_id, kms_key)
   mobile_e164_token VARCHAR(64), -- HMAC-SHA256(e164_phone, kms_key)
   email_token VARCHAR(64), -- HMAC-SHA256(lower(email), kms_key)
   legal_name VARCHAR(200),
   date_of_birth DATE,
   kyc_status VARCHAR(20),
   risk_rating VARCHAR(10),
   residency_country VARCHAR(3),
   source_updated_at TIMESTAMP
);

CREATE TABLE stg_crm_contact (
   crm_contact_id VARCHAR(40) NOT NULL, -- Salesforce Contact Id
   mobile_e164_token VARCHAR(64),
   email_token VARCHAR(64),
   full_name VARCHAR(200),
   marketing_opt_in BOOLEAN,
   source_updated_at TIMESTAMP
);

CREATE TABLE stg_amplitude_user (
   amplitude_user_id VARCHAR(64) NOT NULL, -- Amplitude anonymous/user id
   device_id VARCHAR(64),
   login_bound_mobile_token VARCHAR(64), -- set only after authenticated session
   first_seen_at TIMESTAMP,
   last_seen_at TIMESTAMP
);
```

> **WHY THIS WAY**: Tokenizing at landing, rather than at query time, is what makes ”PII masked for non-privileged users” an enforced property of the pipeline instead of a hope. A Glue job or an analyst querying `stg_core_customer` directly cannot see a real national ID even by accident – the raw value was never written to this table in the first place.

2 Deterministic Waterfall: Passes 1–3

Core banking’s KYC identity is the trust anchor. Every KYC-complete core customer mints an ECID; CRM and Amplitude records only ever attach to an ECID that core banking already created – they never mint one themselves.

The crosswalk itself is a thin, append-mostly bridge table – one row per (source system, source customer ID) pair, exactly the grain described in Deliverable 2’s entity dictionary. It is the one table every pass in this document writes to, so its shape is worth pinning down before the waterfall query itself:

```
CREATE TABLE identity_crosswalk (
   ecid VARCHAR(32) NOT NULL, -- MD5 of the anchoring national_id_token
   source_system VARCHAR(20) NOT NULL, -- 'CORE' | 'CRM' | 'AMPLITUDE'
   source_customer_id VARCHAR(64) NOT NULL, -- source_id from the pass that resolved it
   match_confidence FLOAT,
   match_method VARCHAR(40), -- e.g. DETERMINISTIC_NATIONAL_ID, PROBABILISTIC_FINDMATCHES
   linked_at TIMESTAMP,
   PRIMARY KEY (source_system, source_customer_id)
);
```

```sql
-- ========================================================================
-- Deterministic waterfall. A record is checked against the strongest
-- available signal first; passes are UNION ALL'd, not chained, because
-- each pass targets a disjoint slice of the unmatched population.
-- match_confidence here is retained for audit/reporting granularity
-- only -- every deterministic pass is treated as equally authoritative
-- regardless of its numeric value. The 0.90/0.97 cutoffs used by the
-- probabilistic pass (Section 3) are a separate scale that only ever
-- applies to PROBABILISTIC_FINDMATCHES rows, never compared against
-- these deterministic scores.
-- ========================================================================

WITH core_keyed AS (
   -- Pass 1: every KYC'd core customer gets an ECID from national_id_token
   SELECT
      core_customer_id AS source_id,
      'CORE' AS source_system,
      national_id_token,
      mobile_e164_token,
      email_token,
      'DETERMINISTIC_NATIONAL_ID' AS match_method,
      1.00 AS match_confidence
   FROM stg_core_customer
   WHERE national_id_token IS NOT NULL -- KYC complete
),

crm_matched AS (
   -- Pass 2/3: CRM contact attaches via verified mobile first, else
   -- verified email -- the CASE order mirrors the waterfall's own
   -- trust ordering, so a contact matching on both still records the
   -- stronger signal.
   SELECT
      c.crm_contact_id AS source_id,
      'CRM' AS source_system,
      k.national_id_token,
      CASE WHEN c.mobile_e164_token IS NOT NULL AND c.mobile_e164_token = k.mobile_e164_token
          THEN 'DETERMINISTIC_MOBILE' ELSE 'DETERMINISTIC_EMAIL' END AS match_method,
      CASE WHEN c.mobile_e164_token IS NOT NULL AND c.mobile_e164_token = k.mobile_e164_token
          THEN 0.99 ELSE 0.97 END AS match_confidence
   FROM stg_crm_contact c
   JOIN core_keyed k
     ON (c.mobile_e164_token IS NOT NULL AND c.mobile_e164_token = k.mobile_e164_token)
     OR (c.email_token IS NOT NULL AND c.email_token = k.email_token)
),

amplitude_matched AS (
   -- Amplitude device attaches only once an authenticated session has
   -- bound the device to a verified mobile number -- never speculatively.
   SELECT
      a.amplitude_user_id AS source_id,
      'AMPLITUDE' AS source_system,
      k.national_id_token,
      'DETERMINISTIC_MOBILE' AS match_method,
      0.99 AS match_confidence
   FROM stg_amplitude_user a
   JOIN core_keyed k
     ON a.login_bound_mobile_token = k.mobile_e164_token
),

all_matched AS (
```

```sql
    SELECT source_id, source_system, national_id_token, match_method, match_confidence FROM
        core_keyed
    UNION ALL SELECT source_id, source_system, national_id_token, match_method, match_confidence
        FROM crm_matched
    UNION ALL SELECT source_id, source_system, national_id_token, match_method, match_confidence
        FROM amplitude_matched
)

INSERT INTO identity_crosswalk (ecid, source_system, source_customer_id, match_confidence,
    match_method, linked_at)
SELECT
    MD5(national_id_token) AS ecid, -- Enterprise Customer ID: stable cluster key,
                                    -- derived from the *tokenized* national ID --
                                    -- never the raw value (see Section 1)
    source_system,
    source_id AS source_customer_id,
    match_confidence,
    match_method,
    CURRENT_TIMESTAMP AS linked_at
FROM all_matched;

-- Everything that did NOT resolve here -- CRM leads with no core-banking
-- match at all -- falls through to the probabilistic pass in Section 3.
```

WHY THIS WAY: Notice `crm_matched` joins on mobile *or* email, not name, and records which one actually matched rather than collapsing both into one undifferentiated pass. Walking the Amina example from Deliverable 1 through this query: her core banking row mints `ecid = MD5(national_id_token)` at Pass 1 – computed from her *tokenized* national ID, never the raw `784-1997-...` value itself, which this pipeline never persists past staging (Section 1). Her Salesforce contact has no national ID to compare, so it never even reaches that branch – but it joins into `crm_matched` with `match_method = `DETERMINISTIC_MOBILE'` because `+971501234567` matches on `mobile_e164_token`, *despite* her CRM email (`amina.s92@gmail.com`) and core banking email (`amina.alsuwaidi@icloud.com`) sharing nothing at all. If this query only compared email, or compared name, she would never resolve here – she would fall all the way to Section 3, for no good reason, since a perfectly reliable phone match was available the whole time.

3 Probabilistic Fallback: The Residual Pool

Everything that reaches this section failed every deterministic key in Section 2 – typically a CRM lead who never gave a phone number that matches, or gave one with a typo. Comparing every such lead against every unmatched core customer is a similarity join, not a lookup, and at real scale it needs a blocking key and a proper similarity function – a poor fit for a single SQL statement, and a good fit for a Spark job. This is Glue/PySpark pseudocode, not because the logic couldn't be expressed in SQL, but because doing it well shouldn't be forced into one query. See Deliverable 1, Section 2.2 for the technology comparison (AWS Glue FindMatches, selected) and the feasibility discussion (labeling effort, integration, and why this is deliberately not an LLM-based decision).

```python
# ======================================================================
# AWS Glue job (PySpark). Runs only on the residual pool that failed the
# deterministic waterfall above. Never auto-merges outright: below-
# threshold candidates and the review band both land in a steward queue.
# ======================================================================

from pyspark.sql import functions as F
```

```

unmatched_crm = crm_contacts.join(identity_crosswalk, on="crm_contact_id", how="left_anti")
unmatched_core = core_customers.join(identity_crosswalk, on="core_customer_id", how="left_anti")

# Blocking key keeps this from being an O(n*m) cross join: only compare
# within the same birth year and residency country.
blocked = (
   unmatched_crm.withColumn("block_key", F.concat_ws("_", F.year("date_of_birth"), "
       residency_country"))
   .join(
      unmatched_core.withColumn("block_key", F.concat_ws("_", F.year("date_of_birth"), "
          residency_country")),
      on="block_key"
   )
)

# name_similarity: Jaro-Winkler over normalized (lower, whitespace-
# collapsed) name strings -- tolerant of the kind of formatting drift
# seen between "Amina Al Suwaidi" and "AMINA MOHAMED AL SUWAIDI".
scored = blocked.withColumn(
   "name_similarity", jaro_winkler_udf(F.col("full_name"), F.col("legal_name"))
).withColumn(
   "dob_exact_match", F.col("date_of_birth_crm") == F.col("date_of_birth_core")
)

# Three-way split, per the Fellegi-Sunter framing in Deliverable 1 S2.2:
# confidently the same person, confidently different, or genuinely
# ambiguous and worth a human's attention. Nothing in the middle band
# auto-merges.
auto_merge = scored.filter(
   (F.col("name_similarity") >= 0.97) & F.col("dob_exact_match")
).withColumn("match_method", F.lit("PROBABILISTIC_FINDMATCHES")) \
 .withColumn("match_confidence", F.col("name_similarity"))

steward_review_queue = scored.filter(
   (F.col("name_similarity") >= 0.90) & (F.col("name_similarity") < 0.97) & F.col("
       dob_exact_match")
).withColumn("queue_reason", F.lit("BELOW_AUTO_MERGE_THRESHOLD"))

# auto_merge -> appended to identity_crosswalk, same shape as Section 2
# steward_review_queue -> human review UI (spreadsheet-based at Day 30/60)
# everything scoring below 0.90 -> left unmatched, re-attempted next run,
# never silently discarded
```

On Feedback

Steward decisions made against `steward_review_queue` are periodically folded back in as new labeled training examples for FindMatches, per Deliverable 1 Section 2.2 – the classifier's calibration improves from real corrections over time rather than staying frozen at its first trained version.

4 Survivorship Merge: Publishing the Golden SCD2 Record

Once a cluster's crosswalk rows exist, the golden record is assembled attribute by attribute using the field-level priority rules from Deliverable 1, Section 2.3 – core banking wins identity fields, CRM wins marketing intent, and a genuine disagreement between two authoritative sources is flagged rather than silently resolved.

`dim_customer_golden` is the SCD2 target every survivorship pass publishes to – the same field-level shape as Deliverable 2's `CUSTOMER_GOLDEN` entity, plus the two SCD2 bookkeeping columns. `marketing_opt_`

`in` is the one column here core banking never populates; it always arrives via the `crm` join below, per the survivorship rule that gives CRM sole ownership of marketing intent.

```sql
CREATE TABLE dim_customer_golden (
   ecid VARCHAR(32) NOT NULL,
   legal_name VARCHAR(200),
   date_of_birth DATE,
   national_id_token VARCHAR(64),
   mobile_e164_token VARCHAR(64),
   email_token VARCHAR(64),
   marketing_opt_in BOOLEAN,
   kyc_status VARCHAR(20),
   risk_rating VARCHAR(10),
   residency_country VARCHAR(3),
   scd_valid_from TIMESTAMP,
   scd_valid_to TIMESTAMP,
   scd_is_current BOOLEAN
);
```

```sql
-- ========================================================================
-- Survivorship: one candidate golden row per ECID, built attribute-by-
-- attribute. CORE (regulated/KYC) beats CRM (self-declared) beats
-- nothing -- never "pick whichever record is newest" wholesale.
-- ========================================================================

CREATE OR REPLACE VIEW v_golden_customer_candidate AS
SELECT
   xw_core.ecid,
   core.legal_name, -- CORE only: legal identity
   core.date_of_birth, -- CORE only
   core.national_id_token, -- CORE only
   COALESCE(core.mobile_e164_token, crm.mobile_e164_token) AS mobile_e164_token,
   COALESCE(core.email_token, crm.email_token) AS email_token,
   core.kyc_status,
   core.risk_rating,
   core.residency_country,
   COALESCE(crm.marketing_opt_in, FALSE) AS marketing_opt_in,
   GREATEST(core.source_updated_at, COALESCE(crm.source_updated_at, core.source_updated_at))
                                          AS source_updated_at
FROM identity_crosswalk xw_core
JOIN stg_core_customer core
  ON xw_core.source_system = 'CORE' AND xw_core.source_customer_id = core.core_customer_id
LEFT JOIN identity_crosswalk xw_crm
  ON xw_crm.ecid = xw_core.ecid AND xw_crm.source_system = 'CRM'
LEFT JOIN stg_crm_contact crm
  ON xw_crm.source_customer_id = crm.crm_contact_id;

-- SCD Type 2: close out the current row only if a tracked attribute
-- actually changed; otherwise leave history untouched.
MERGE INTO dim_customer_golden AS tgt
USING v_golden_customer_candidate AS src
ON tgt.ecid = src.ecid AND tgt.scd_is_current = TRUE
WHEN MATCHED AND (
      tgt.legal_name, tgt.kyc_status, tgt.risk_rating, tgt.mobile_e164_token, tgt.email_token
    ) IS DISTINCT FROM (
      src.legal_name, src.kyc_status, src.risk_rating, src.mobile_e164_token, src.email_token
    )
THEN UPDATE SET
   scd_valid_to = CURRENT_TIMESTAMP,
   scd_is_current = FALSE
WHEN NOT MATCHED THEN
```

```
INSERT (ecid, legal_name, date_of_birth, national_id_token, mobile_e164_token, email_token,
      kyc_status, risk_rating, residency_country, marketing_opt_in,
      scd_valid_from, scd_valid_to, scd_is_current)
VALUES (src.ecid, src.legal_name, src.date_of_birth, src.national_id_token, src.mobile_e164_token
    , src.email_token,
      src.kyc_status, src.risk_rating, src.residency_country, src.marketing_opt_in,
      CURRENT_TIMESTAMP, NULL, TRUE);

-- A closed-out row's replacement is inserted in the same pass, keyed off
-- the just-closed ecids -- the standard SCD2 "expire old, insert new"
-- pair, omitted here as a second, identical-shaped INSERT for brevity.
```

5 Validating the Merge

Two checks worth running after every identity resolution batch, before anything downstream trusts the result.

```
-- Check 1: no ECID should ever span more than one national_id_token --
-- if this returns rows, two genuinely different core-banking customers
-- were merged, which is the one failure mode this design treats as
-- worse than a missed match.
SELECT ecid, COUNT(DISTINCT national_id_token) AS distinct_nids
FROM stg_core_customer core
JOIN identity_crosswalk xw
  ON xw.source_system = 'CORE' AND xw.source_customer_id = core.core_customer_id
GROUP BY ecid
HAVING COUNT(DISTINCT national_id_token) > 1;

-- Check 2: confirm the worked example actually resolved across all
-- three sources into one ECID, the way Section 2's walkthrough claims.
SELECT source_system, source_customer_id, match_method, match_confidence
FROM identity_crosswalk
WHERE ecid = (
   SELECT ecid FROM identity_crosswalk
   WHERE source_system = 'CORE' AND source_customer_id = 'CIF-000481932'
)
ORDER BY linked_at;
-- Expected: three rows, all sharing one ecid -- CORE / CIF-000481932
-- (DETERMINISTIC_NATIONAL_ID), CRM / (Amina's contact id)
-- (DETERMINISTIC_MOBILE), and AMPLITUDE / (her device id)
-- (DETERMINISTIC_MOBILE).
```

> **WHY THIS WAY**: Check 1 is the one query this whole design exists to make pass. Everything in Deliverable 1 Section 2 – the trust-ordered waterfall, the conservative probabilistic thresholds, never letting Amplitude originate a link – is ultimately in service of this single invariant never being violated: one national ID, one ECID, always.